Package 'TriSfSVD'

May 26, 2026

Type Package

Title Functional SVD Based Tri-Clustering

Version 0.1.0

Author Yue Zhao [aut, cre],
Thierry Chekouo [aut],
Sandra Safo [aut]

Maintainer Yue Zhao <zhao0646@umn.edu>

Description Implements sparse functional singular value decomposition methods for data-driven identification of biclustering and triclustering structures in high-dimensional sparse multivariate functional data. Provides functions for estimating sparse functional components, identifying sample clusters, variable clusters, and time-domain subregions, and reconstructing interpretable sample-feature-time patterns.

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.1.0)

Imports energy,
irlba,
Matrix,
parallel,
softImpute,
stats,
utils

R topics documented:

genData *Generate Sparse Multivariate Functional Data*

Description

Generates sparse multivariate functional data with known non-overlapping sample clusters and feature clusters for illustrating and evaluating Trisfsvd.

Usage

```
genData(
  n_samples = 40,
  true_rank = 4,
  n_features = 10,
  domain_points = 30,
  noise_sd = 0.2,
  missing_rate = 0.5,
  signal_means = NULL,
  signal_sd = 0.02
)
```

Arguments

n_samples	Number of samples.
true_rank	Number of latent sparse functional components used to generate the data.
n_features	Number of functional variables.
domain_points	Number of observed domain points per functional variable.
noise_sd	Standard deviation of the additive Gaussian noise.
missing_rate	Proportion of within-curve observations replaced by NA.
signal_means	Optional vector controlling the component-specific signal strengths. If omitted, a decreasing default sequence is used.
signal_sd	Standard deviation used to generate nonzero sample scores.

Details

The generator is designed to mirror the structure targeted by Trisfsvd. Samples are divided into non-overlapping latent groups, and functional variables are assigned to component-specific active sets. The output is returned in the nested-list format expected by the fitting and post-processing functions in the package.

Value

A list with components:

X_list	Complete nested-list data without missing values.
X_list_na	Nested-list data with missing values inserted at random.
X_mat	Underlying matrix representation of the simulated data.
U	Matrix of true sample scores.

`V`	Matrix of true functional loadings.
`D`	Diagonal matrix of component strengths.
`time_grid`	Common domain grid on $[0, 1]$.
`subject_groups`	List of true non-overlapping sample clusters.
`feature_groups`	List of true component-specific feature clusters.
`feature_group_sizes`	Numbers of active features assigned to each component.
`sample_group_sizes`	Numbers of active samples assigned to each component.

Examples

```
set.seed(2)

sim <- genData()

str(sim$X_list_na, max.level = 2)
sim$feature_group_sizes
sim$sample_group_sizes

sim2 <- genData(
  n_samples = 60,
  true_rank = 4,
  n_features = 20,
  noise_sd = 0.3
)

sim2$feature_group_sizes
```

`getClusterAssoc` *Refine Sample Clusters and Summarize Cluster Associations*

Description

Post-processes fitted `Trisfsvd` components to refine sample clusters and quantify their association with component-specific feature clusters and selected time-domain subregions.

Usage

```
getClusterAssoc(
  fit, X, n_clusters, component_indices = NULL, scale_u = TRUE,
  km_iter_max = 10000, km_nstart = 1000, soft_rank = 1, soft_lambda = 0,
  dcor_R = 199, dcor_index = 1, bootstrapping = FALSE,
  bootstrap_B = 1000, bootstrap_seed = 2025, bootstrap_n_min = 5,
  keep_boot_array = FALSE,
  mask_align = c("strict", "truncate", "pad_false"),
  block_align = c("strict", "pad", "truncate"),
  block_pad_value = NA_real_
)
```

Arguments

fit	A fitted object returned by Trisfsvd.
X	The original nested-list functional data used for fitting. Missing values may be represented with NA.
n_clusters	Number of refined sample clusters to estimate by k-means.
component_indices	Integer vector specifying which fitted components to use. By default, the first min(length(fit$results_list), n_clusters) components are used.
scale_u	Logical indicating whether to standardize the selected sample scores before k-means clustering.
km_iter_max	Maximum number of iterations for k-means.
km_nstart	Number of random starts for k-means.
soft_rank	Rank used in the **softImpute** decomposition of the masked data matrices.
soft_lambda	Regularization parameter passed to softImpute::softImpute().
dcor_R	Number of permutations used in energy::dcor.test().
dcor_index	Index parameter passed to energy::dcor.test().
bootstrapping	Logical indicating whether to compute bootstrap summaries for the distance correlation matrix.
bootstrap_B	Number of bootstrap resamples when bootstrapping = TRUE.
bootstrap_seed	Random seed used for the bootstrap resampling step.
bootstrap_n_min	Minimum number of finite paired observations required when computing bootstrap distance correlations.
keep_boot_array	Logical indicating whether to retain the full bootstrap array internally.
mask_align	Alignment rule used when masking selected subregions.
block_align	Alignment rule used when converting masked curves to a block matrix.
block_pad_value	Padding value used when block_align = "pad".

Details

The function is intended for settings where the fitted u vectors from Trisfsvd are informative but not sufficiently distinct to define final sample clusters on their own. It first forms a low-dimensional sample representation by stacking selected u vectors, then applies k-means to obtain refined sample clusters.

For each selected component, the fitted loading functions are converted into feature-specific subregion masks. These masks are applied to the original functional data, and a rank-reduced summary is obtained by **softImpute**. Distance covariance and distance correlation summaries are then used to quantify the association between the refined sample clusters and the component-specific functional patterns.

Value

A list with components:

sample_cluster	A list of sample-index vectors defining the refined sample clusters.

`feature_cluster`
: A list of feature-index vectors defining the component-specific feature clusters taken from the fitted `Trisfsvd` object.

`dcov_stat_matrix`
: Distance covariance test statistic matrix.

`dcor_hat`
: Bootstrap target distance correlation matrix. Returned only when `bootstrapping = TRUE`.

`ci_lo`
: Lower 95% bootstrap confidence bound matrix. Returned only when `bootstrapping = TRUE`.

`ci_hi`
: Upper 95% bootstrap confidence bound matrix. Returned only when `bootstrapping = TRUE`.

`se_hat`
: Bootstrap standard error matrix. Returned only when `bootstrapping = TRUE`.

Examples

```
## Not run:
set.seed(2)

sim <- genData(
  n_samples = 20,
  true_rank = 2,
  n_features = 6,
  domain_points = 20,
  noise_sd = 0.15,
  missing_rate = 0.3
)

fit <- Trisfsvd(
  X = sim$X_list_na,
  k = 2,
  gamma_candidates = c(0, 0.05),
  theta_candidates = c(0, 0.05),
  lambda_candidates = c(0, 0.1),
  alpha_candidates = c(0, 1),
  max_outer_iter = 2,
  tol_outer = 5e-3,
  verbose = FALSE,
  use_sgl = TRUE,
  max_iter = 500,
  eps = 1e-3,
  gamma_adaptive_power = 0.5,
  theta_adaptive_power = 0,
  lambda_adaptive_power = 0,
  extend_weight_domain = 0.5,
  extend_weight_smooth = 0,
  extend_weight_group = 0,
  extend_weight_u = 0.5,
  parallel = FALSE,
  backtracking = TRUE,
  subj_overlap = TRUE,
  var_overlap = TRUE,
  subreg_overlap = TRUE
)

assoc <- getClusterAssoc(
```

```
  fit = fit,
  X = sim$X_list_na,
  n_clusters = 2,
  component_indices = 1:2,
  bootstrapping = FALSE
)

assoc$sample_cluster
assoc$dcov_stat_matrix

## End(Not run)
```

---

plotResult *Plot Sample-Feature-Time Patterns for Paired Clusters*

Description

Plots observed and reconstructed functional patterns for paired sample and feature clusters obtained either directly from Trisfsvd or from getClusterAssoc.

Usage

```
plotResult(
  fit, X, sample_cluster, feature_cluster, association_indices = NULL,
  mean = TRUE, use_s = TRUE, show_variables = TRUE, random_seed = NULL,
  ranks = NULL, nrow_page = 4, ncol_page = 5, colours = NULL
)
```

Arguments

fit	A fitted object returned by Trisfsvd.
X	The original nested-list functional data. Each sample is a list of functional variables, and each variable is a numeric domain vector.
sample_cluster	A list of sample-index vectors, such as fit$sample_cluster or the sample_cluster output from getClusterAssoc.
feature_cluster	A list of feature-index vectors, such as fit$feature_cluster or the feature_cluster output from getClusterAssoc.
association_indices	Integer vector indicating which paired associations to plot. By default, all available pairs up to min(length(sample_cluster), length(feature_cluster)) are shown.
mean	Logical indicating whether to plot cluster mean curves. If TRUE, the mean observed curve and mean reconstructed curve are shown. If FALSE, all observed sample curves and all reconstructed sample curves are shown.
use_s	Logical indicating whether to include the fitted scalar values s in the reconstructed functional curves.
show_variables	Controls how many variables are displayed for each association. If TRUE, all variables in the feature cluster are shown. If a single integer is supplied, that many variables are randomly selected from the feature cluster.

random_seed	Optional random seed used when show_variables is a single integer.
ranks	Integer vector of fitted ranks to use in the reconstruction. By default, all fitted components in fit$results_list are used.
nrow_page	Number of panel rows per plotting page.
ncol_page	Number of panel columns per plotting page.
colours	Optional vector of colours used for the paired associations.

Details

For each selected functional variable, the function reconstructs the fitted sample-feature-time pattern by summing the chosen sparse functional components. It then subsets the requested sample cluster and overlays the observed functional trajectories with the corresponding reconstructed trajectories. This provides a direct visual summary of how the fitted sparse functional decomposition represents the paired sample and feature clusters.

Value

Invisibly returns a list giving the feature indices plotted for each association.

Examples

```
## Not run:
set.seed(2)

sim <- genData(
  n_samples = 20,
  true_rank = 2,
  n_features = 6,
  domain_points = 20,
  noise_sd = 0.15,
  missing_rate = 0.3
)

fit <- Trisfsvd(
  X = sim$X_list_na,
  k = 2,
  gamma_candidates = c(0, 0.05),
  theta_candidates = c(0, 0.05),
  lambda_candidates = c(0, 0.1),
  alpha_candidates = c(0, 1),
  max_outer_iter = 2,
  tol_outer = 5e-3,
  verbose = FALSE,
  use_sgl = TRUE,
  max_iter = 500,
  eps = 1e-3,
  gamma_adaptive_power = 0.5,
  theta_adaptive_power = 0,
  lambda_adaptive_power = 0,
  extend_weight_domain = 0.5,
  extend_weight_smooth = 0,
  extend_weight_group = 0,
  extend_weight_u = 0.5,
  parallel = FALSE,
  backtracking = TRUE,
```

```
  subj_overlap = TRUE,
  var_overlap = TRUE,
  subreg_overlap = TRUE
)

plotResult(
  fit = fit,
  X = sim$X_list_na,
  sample_cluster = fit$sample_cluster,
  feature_cluster = fit$feature_cluster,
  mean = TRUE,
  show_variables = 3
)

## End(Not run)
```

Trisfsvd *Estimate Sparse Functional Components and Tri-Clusters*

Description

Estimates sparse functional singular value decomposition components for data-driven identification of biclustering and triclustering structure in high-dimensional sparse multivariate functional data.

Usage

```
Trisfsvd(
  X, k = 1, gamma_candidates, alpha_candidates, theta_candidates,
  lambda_candidates = NULL, max_outer_iter = 20, tol_outer = 1e-5,
  use_sgl = TRUE, verbose = TRUE, L_init = NULL, max_iter = 30000,
  eps = 1e-5, gamma_adaptive_power = 1, theta_adaptive_power = 1,
  lambda_adaptive_power = 1, extend_weight_domain, extend_weight_smooth,
  extend_weight_group, subj_overlap = FALSE, var_overlap = FALSE,
  subreg_overlap = FALSE, parallel = FALSE, n_cores = detectCores() - 1,
  extend_weight_u, backtracking
)
```

Arguments

X	A nested list of sparse multivariate functional observations. The outer list indexes samples. Each sample is a list of functional variables, and each variable is a numeric vector of domain measurements. Missing values should be represented with NA.
k	Number of rank-1 components to extract.
gamma_candidates	Candidate tuning values for sample-level sparsity.
alpha_candidates	Candidate tuning values for smoothness.
theta_candidates	Candidate tuning values for variable-level sparsity.
lambda_candidates	Candidate tuning values for domain-level sparsity.

`max_outer_iter` Maximum number of outer iterations.

`tol_outer` Convergence tolerance for the iterative algorithm used to estimate each rank-one regularized SVD component.

`use_sgl` Logical indicating whether to use the sparse-group-lasso style update. If `TRUE`, sparse-group-lasso is used to obtain tri-cluster results. If `FALSE`, group-lasso is used to obtain bicluster results.

`verbose` Logical indicating whether to print progress.

`L_init` Initial step size parameter for the FISTA backtracking procedure.

`max_iter` Maximum number of iterations for the sparse-group-lasso or group-lasso updates.

`eps` Convergence tolerance for the FISTA iterations. The algorithm stops when the change between two successive iterates is smaller than `eps`.

`gamma_adaptive_power`
Adaptive weighting power for `gamma_candidates`.

`theta_adaptive_power`
Adaptive weighting power for variable-level sparsity tuning.

`lambda_adaptive_power`
Adaptive weighting power for domain-level sparsity tuning.

`extend_weight_domain`
Weights in the EBIC criterion for domain-level sparsity tuning selection.

`extend_weight_smooth`
Weights in the EBIC criterion for smoothing tuning selection.

`extend_weight_group`
Weights in the EBIC criterion for variable-level sparsity tuning selection.

`subj_overlap` Logical indicating whether samples may overlap across estimated tri-clusters or biclusters.

`var_overlap` Logical indicating whether variables may overlap across estimated tri-clusters or biclusters.

`subreg_overlap` Logical indicating whether domain-level regions may overlap across estimated tri-clusters or biclusters.

`parallel` Logical indicating whether to use parallel computation.

`n_cores` Number of cores used when `parallel = TRUE`.

`extend_weight_u`
Weights in the EBIC criterion for sample-level sparsity tuning selection.

`backtracking` Logical indicating whether to use backtracking in the FISTA algorithm. If `TRUE`, the backtracking FISTA algorithm is used. If `FALSE`, FISTA is run without backtracking.

Details

The function takes sparse multivariate functional observations stored as a nested list and extracts rank-one sparse functional components. For each component, the fitted sample scores, selected variables, and selected time-domain subregions define an interpretable bicluster or tricluster pattern, depending on the chosen penalty structure.

Typical use consists of three steps: prepare the nested-list input, fit `Trisfsvd()`, and then inspect the returned sample clusters, feature clusters, loading functions, and reconstruction summaries. When `parallel = TRUE`, tuning-parameter path evaluations may use multiple cores through the **parallel** package.

Value

A list with components:

`results_list` A list of fitted rank-1 components. Each component contains estimated sample scores, variable/domain loading objects, selected tuning parameters, and related model-selection output.

`sample_cluster` A list of sample-index vectors. Each element contains the sample indices whose fitted u values are nonzero for one extracted component.

`feature_cluster`
A list of feature-index vectors. Each element contains the feature indices whose fitted `varphi` functions are not identically zero for one extracted component.

`variance_explained`
Numeric vector giving the cumulative proportion of variation explained by the extracted components.

`BIC_score` Numeric vector of BIC scores for the extracted components.

Examples

```
## Not run:
## ------------------------------------------------------------
## Example: generate data, then fit TriSfSVD
## ------------------------------------------------------------
## The package provides a simulation helper that generates a sparse
## multivariate functional data set in the nested-list format needed
## by Trisfsvd.

set.seed(2)

## Generate a synthetic data set using the package default settings.
## Here this means:
##   - 40 samples
##   - 4 latent rank-one components
##   - 10 functional features
##   - 30 domain points per feature
##   - Gaussian noise with standard deviation 0.2
##   - 50
sim <- genData(
  n_samples = 40,
  true_rank = 4,
  n_features = 10,
  domain_points = 30,
  noise_sd = 0.2,
  missing_rate = 0.5,
  signal_means = NULL,
  signal_sd = 0.02
)

## The returned object contains:
##   - sim$X_list: complete data without missing values
##   - sim$X_list_na: nested-list data with missing values
##   - sim$U, sim$V, sim$D: underlying latent signal objects
##   - sim$sample_group_sizes and sim$feature_group_sizes:
##     the true non-overlapping sample and feature allocations

## Candidate tuning grids. Larger grids may improve tuning resolution
```

```
## but increase computation time.
gamma_grid <- c(0, 0.05, 0.5)
theta_grid <- c(0, 0.01, 0.1)
lambda_grid <- c(0, 0.1, 1)
alpha_grid <- c(0, 1, 10)

## Fit the model. Setting use_sgl = TRUE uses sparse-group-lasso
## updates to estimate tri-cluster structure.
res <- Trisfsvd(
  X = sim$X_list_na,
  k = 2,
  gamma_candidates = gamma_grid,
  theta_candidates = theta_grid,
  lambda_candidates = lambda_grid,
  alpha_candidates = alpha_grid,
  max_outer_iter = 3,
  tol_outer = 2e-3,
  verbose = TRUE,
  use_sgl = TRUE,
  max_iter = 2000,
  eps = 1e-4,
  theta_adaptive_power = 0,
  lambda_adaptive_power = 0,
  gamma_adaptive_power = 0.5,
  extend_weight_domain = 0.5,
  extend_weight_u = 0.5,
  extend_weight_smooth = 0,
  extend_weight_group = 0,
  subj_overlap = TRUE,
  var_overlap = TRUE,
  subreg_overlap = TRUE,
  parallel = FALSE,
  backtracking = TRUE
)

res$sample_cluster
res$feature_cluster

## End(Not run)
```

Index